



U1 INV. FUNDAMENTOS FASTAPI

May/30/2026



cesun
Universidad

CARLOS LEONEL

DESARROLLO DE SOFTWARE BACKEND

Bailon Yira

¿Qué es FastAPI?

Es un framework moderno para desarrollar APIs y aplicaciones web con Python. Fue diseñado para crear servicios web de alto rendimiento, utilizando las características modernas de Python como las anotaciones de tipos (*type hints*). Destaca por su alta velocidad de ejecución, validación automática de datos y su fácil integración de datos y librerías modernas.

Usos de FastAPI

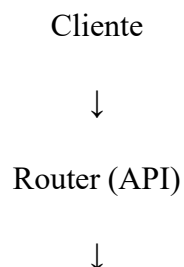
FastAPI se utiliza para:

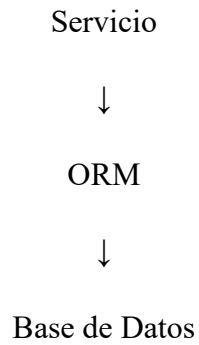
- Desarrollo de APIs REST.
- Microservicios.
- Aplicaciones empresariales.
- Backend para aplicaciones móviles.
- Sistemas de inteligencia artificial.
- Plataformas de comercio electrónico.
- Aplicaciones en la nube.

Arquitectura de FastAPI

- 1. Routers:** Reciben las peticiones HTTP.
- 2. Servicios:** Contienen la lógica de negocio.
- 3. Modelos:** Representan los datos.
- 4. Base de datos:** Almacena la información.

Arquitectura típica





Librerías más utilizadas con FastAPI

- Pydantic
- SQLAlchemy
- SQLModel
- Alembic
- Pytest
- Uvicorn

¿Qué es ORM?

es una técnica que permite trabajar con bases de datos utilizando objetos de Python en lugar de escribir consultas SQL manualmente.

SQL

```
SELECT * FROM usuarios;
```

Se puede hacer:

Python

```
usuarios = session.query(Usuario).all()
```

Comunicación y Validaciones

FastAPI y Asincronía

FastAPI permite utilizar las palabras reservadas:

La asincronía permite que el servidor atienda varias solicitudes mientras espera respuestas externas como:

- consultas a bases de datos,
- llamadas a APIs,
- lectura de archivos.

Ventajas

- Mayor rendimiento.
- Menor tiempo de respuesta.
- Mejor aprovechamiento de recursos.
- Capacidad para manejar miles de conexiones simultáneas.

Por ello FastAPI suele ser más rápido que frameworks tradicionales cuando existen muchas operaciones de entrada y salida.

Uvicorn

Uvicorn es un servidor ASGI (*Asynchronous Server Gateway Interface*).

¿Qué es ASGI?

ASGI es el estándar moderno para aplicaciones Python asíncronas.

FastAPI fue diseñado sobre ASGI y por ello necesita ejecutarse en un servidor compatible como Uvicorn.

Ejemplo:

```
uvicorn main:app --reload
```

¿Por qué FastAPI lo requiere?

Porque procesa solicitudes asíncronas, permite alto rendimiento y maneja múltiples conexiones concurrentes.

Pydantic

Pydantic permite validar automáticamente los datos enviados por los usuarios.

Ejemplo:

```
from pydantic import BaseModel

class Usuario(BaseModel):
    nombre: str
    edad: int
```

Beneficios

- Validación automática.
- Conversión de tipos.
- Menos errores.
- Mayor seguridad.
- Documentación automática.

Persistencia de Datos

Los ORM resuelven el problema de traducir objetos de Python a registros de bases de datos y viceversa.

Ventajas sobre SQL directo

- Menos código repetitivo.
- Mayor seguridad.
- Portabilidad entre bases de datos.
- Mejor mantenimiento.
- Código más legible.

Ejemplo:

Sin ORM:

```
INSERT INTO usuarios(nombre)
VALUES('Ana');
```

Con ORM:

```
usuario = Usuario(nombre="Ana")
session.add(usuario)
```

Alembic (Migraciones)

Alembic permite administrar cambios en la estructura de una base de datos sin perder información.

Sin Alembic:

- Debes modificar la base manualmente.
- Existe riesgo de errores o pérdida de datos.

Con Alembic:

- Se genera una migración.
- La modificación se aplica automáticamente.
- Los datos existentes permanecen intactos.

Beneficios

- Control de versiones de la base de datos.
- Cambios seguros.
- Trabajo colaborativo.
- Historial de modificaciones.

Pytest

Pytest permite automatizar pruebas del software.

¿Por qué probar los endpoints antes de liberar el software?

Porque ayuda a verificar que:

- Las rutas funcionen correctamente.

- Las respuestas sean válidas.
- No existan errores inesperados.
- Las actualizaciones no rompan funcionalidades existentes.

Referencias

IA